Profiling Report

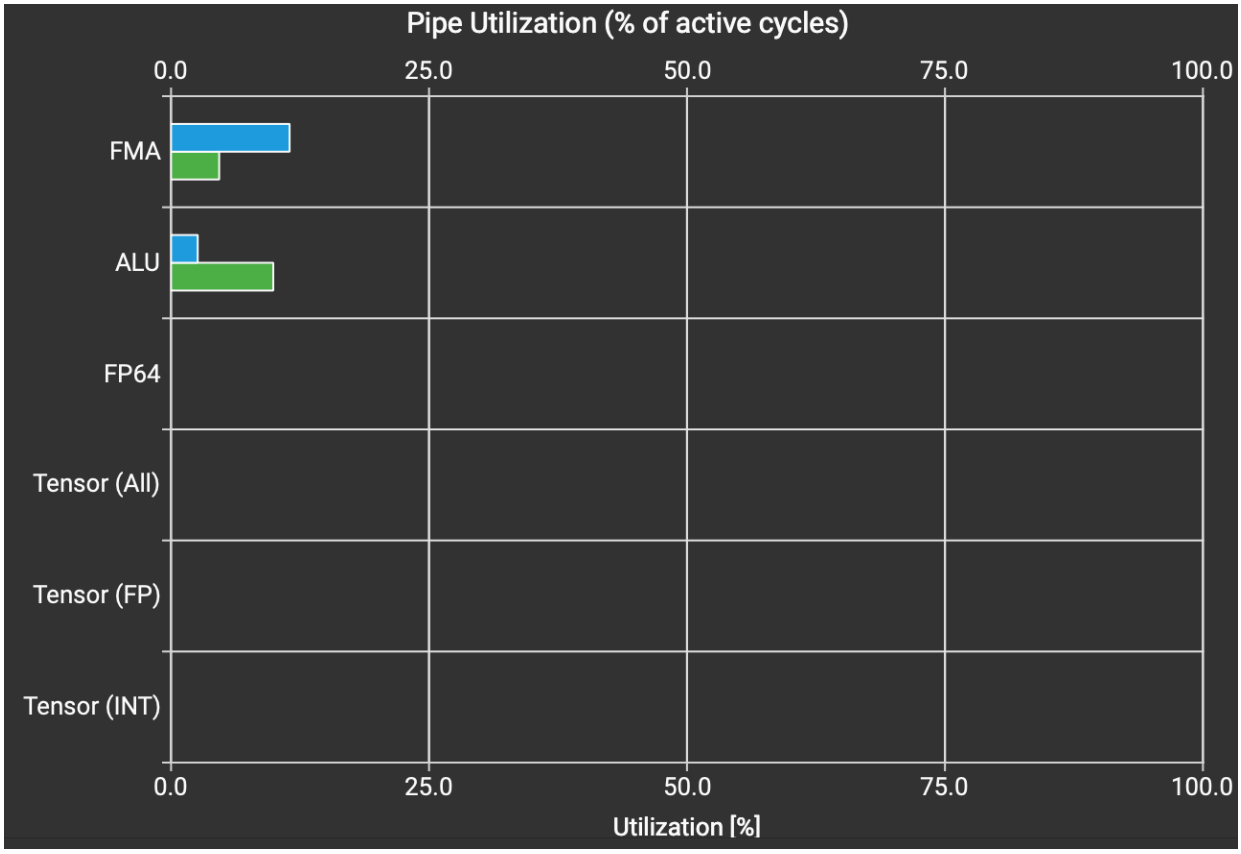
Baseline

The top three time consuming kernels in our baseline implementation are:

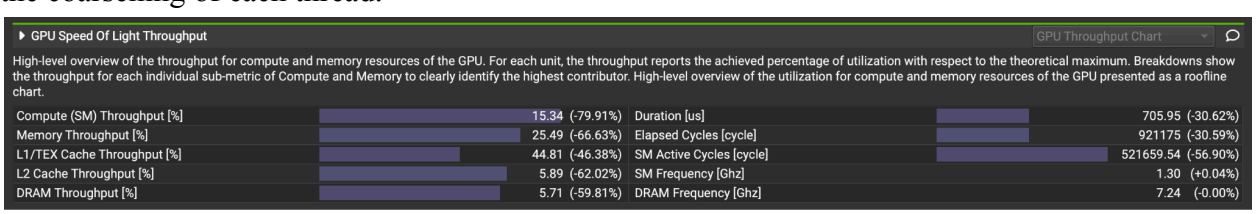
1. (115.9 microseconds) matmul_no_transpose_kernel: this is the naive implementation of matmul which does not transpose its result, implemented in attention
2. (32.9 microseconds) Softmax_forward
3. (30.1 microseconds) layernorm_forward

It is of note that our baseline implementation already leveraged shared memory tiling because the kernel was too slow to perform any tests without it. One obvious area for improvement is that memory accesses for the naive matmul implementation are uncoalesced in our baseline implementation, resulting in much slower runs with higher memory utilization. In the matmul kernel run with the longest duration, 64% of the accessed sectors were excessive, illustrating the amount of memory throughput which could be saved by coalescing accesses.

Opt #0



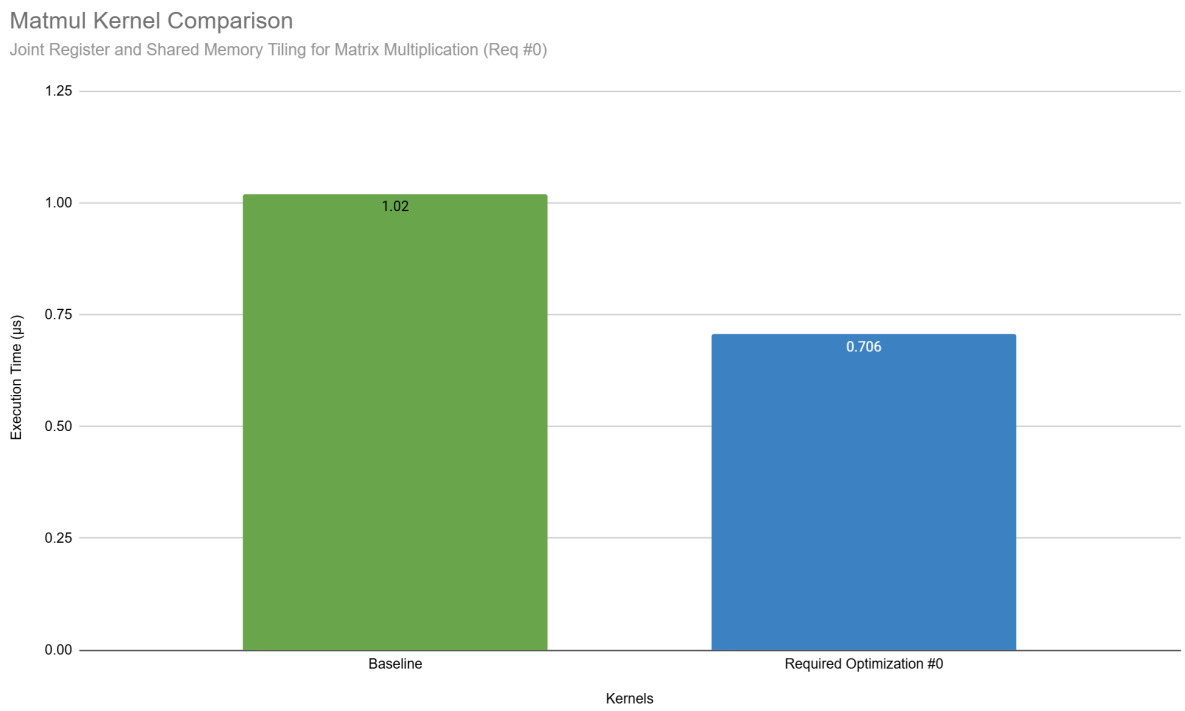
Here we can see a drop in ALU utilization and an associated increase in FMA utilization when compared to the baseline implementation (baseline is green, and optimized kernel is blue). Since we make each thread do more computation, it is reasonable that we have an increase in the FMA. ALU decrease is due to having fewer threads activated for the optimized kernel because of the coarsening of each thread.



GPU Speed Of Light Throughput — GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

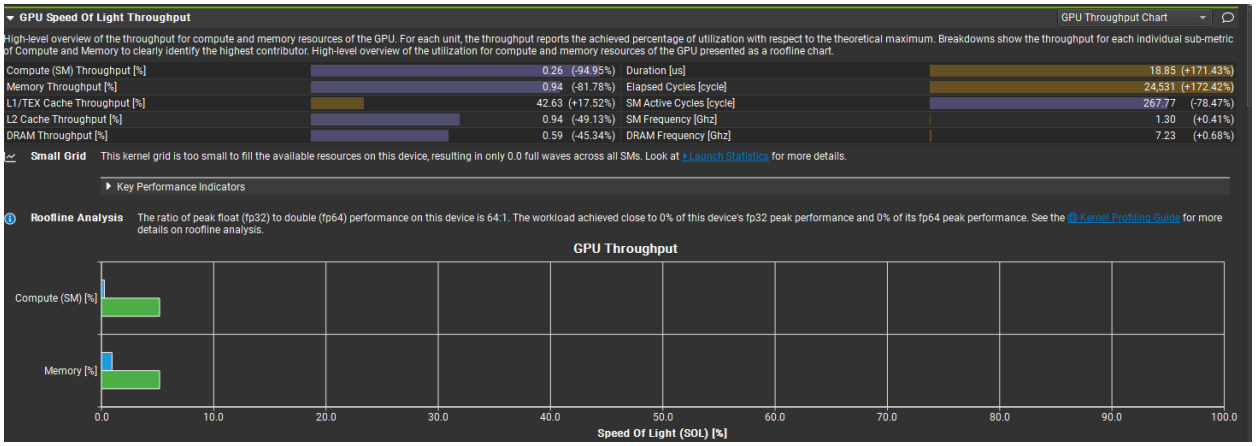
Compute (SM) Throughput [%]	15.34 (-79.91%)	Duration [us]	705.95 (-30.62%)
Memory Throughput [%]	25.49 (-66.63%)	Elapsed Cycles [cycle]	921175 (-30.59%)
L1/TEX Cache Throughput [%]	44.81 (-46.38%)	SM Active Cycles [cycle]	521659.54 (-56.90%)
L2 Cache Throughput [%]	5.89 (-62.02%)	SM Frequency [Ghz]	1.30 (+0.04%)
DRAM Throughput [%]	5.71 (-59.81%)	DRAM Frequency [Ghz]	7.24 (-0.00%)



Comparing the duration of the same matmul kernel between this optimization and the baseline, we can see that, for large matmul operations, this optimization successfully decreases duration by ~30% (We picked the kernel launching 3811 to compare between the baseline and the optimized kernel).

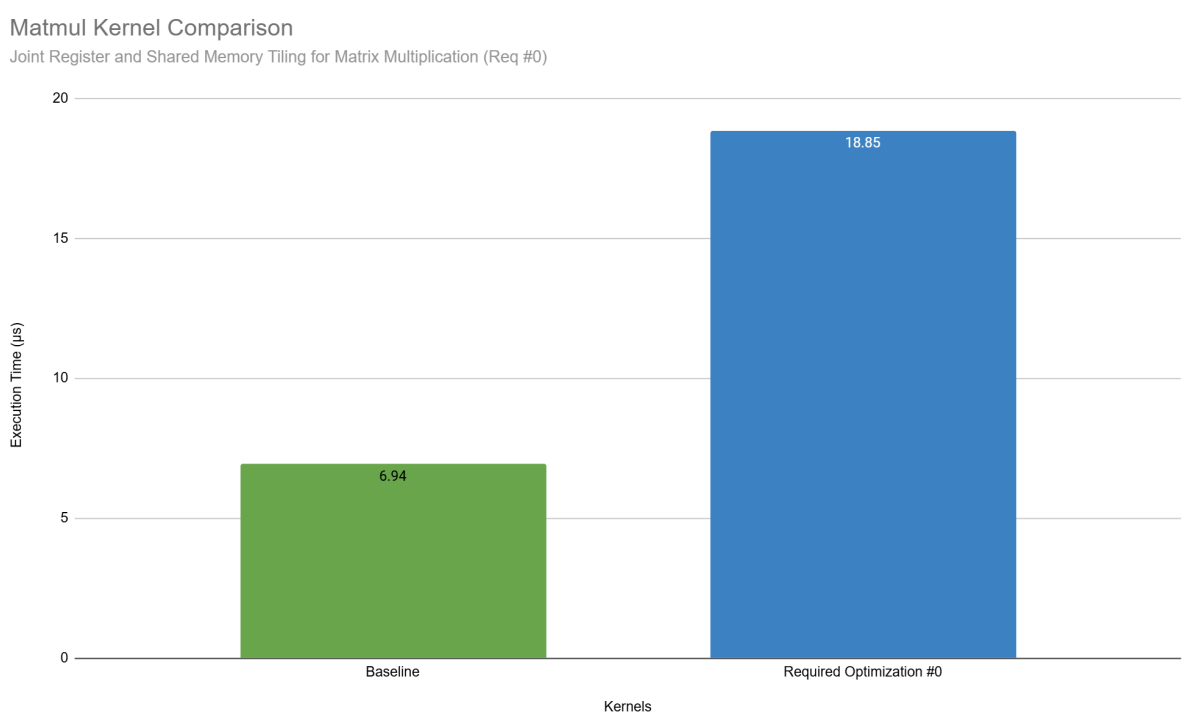
Additionally, we can see that SM throughput is cut in half, and memory throughput is cut by a quarter. We also observe that the matmul kernel in this optimization achieves an occupancy of 4.17% which is much lower than the baseline's occupancy which is 71.9%. This is because we make the parameters for each thread's computing too many outputs for this matrix multiplication, causing less threads activated. With further tuning of the parameter (make each thread do less work), we can find a suitable parameter to higher the compute throughput and occupancy for better utilization.

The interesting thing we found is that the average warps stalled waiting for memory operations per issue cycle for optimized kernels is much smaller than the baseline for both large matrix multiplication and small matrix multiplication (optimized kernel is below 1, the baseline kernel is around 7). We think that this is caused by launching too few threads per block, which also means less warps per block. There are not many warps available for stalling.



	Report	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	Require0	2810 - matmul_forward_kernel	(1, 1, 1)x(8, 8, 1)	18.85 us	24,531	0 - NVIDIA A40	1.30 Ghz	[844828] test_gpt2	
Baseline 1	Baseline	2810 - matmul_forward_kernel	(4, 4, 1)x(16, 16, 1)	6.94 us	9,005	0 - NVIDIA A40	1.30 Ghz	[1642950] test_gpt2	

We also compared the kernel launching 2810, which is a small matrix multiplication (blue is the optimized kernel and green is the baseline). Surprisingly, the optimized kernel actually performed worse than the baseline. From the size, we can tell that the optimized kernel only launched one thread block with 64 threads.



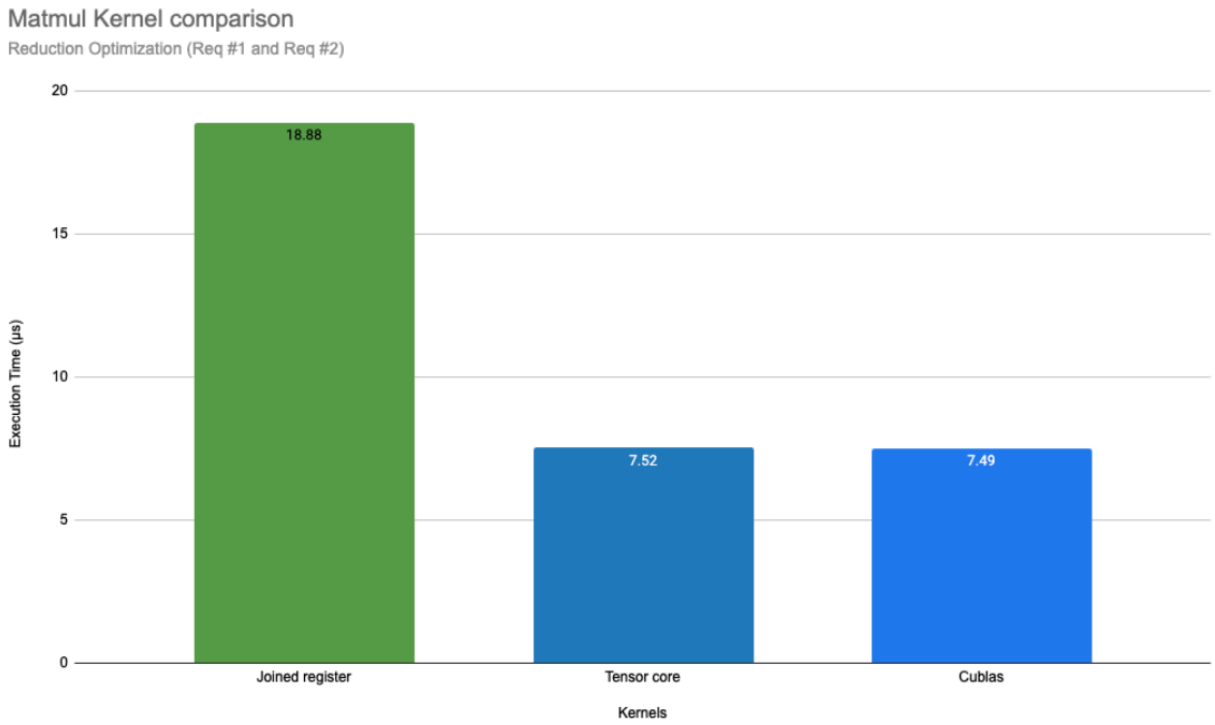
Following is the reason why the optimized version is slower for small matrix multiplication. The optimized kernel initializes 64 accumulator registers per thread with bias values, which is hard to amortize for a small workload. Since each thread is doing more work,

fewer threads per block, this causes less thread-level parallelism (also lower compute throughput).

Following is the reason why the optimized version is faster for large matrix multiplication. The optimized kernel requires less synchronization for threads, which results in less overhead on synchronization. More register data reuse in the kernel to cover the overhead of the longer loading process (from global to shared memory, then to the registers). Since the unroll, we gained better instruction-level parallelism. Also, reduced the shared memory pressure by letting fewer threads compete for the shared memory data.

Opt #1

In this optimization, we leveraged Tensor Core through Nvidia's WMMA API to optimize GEMM operations.

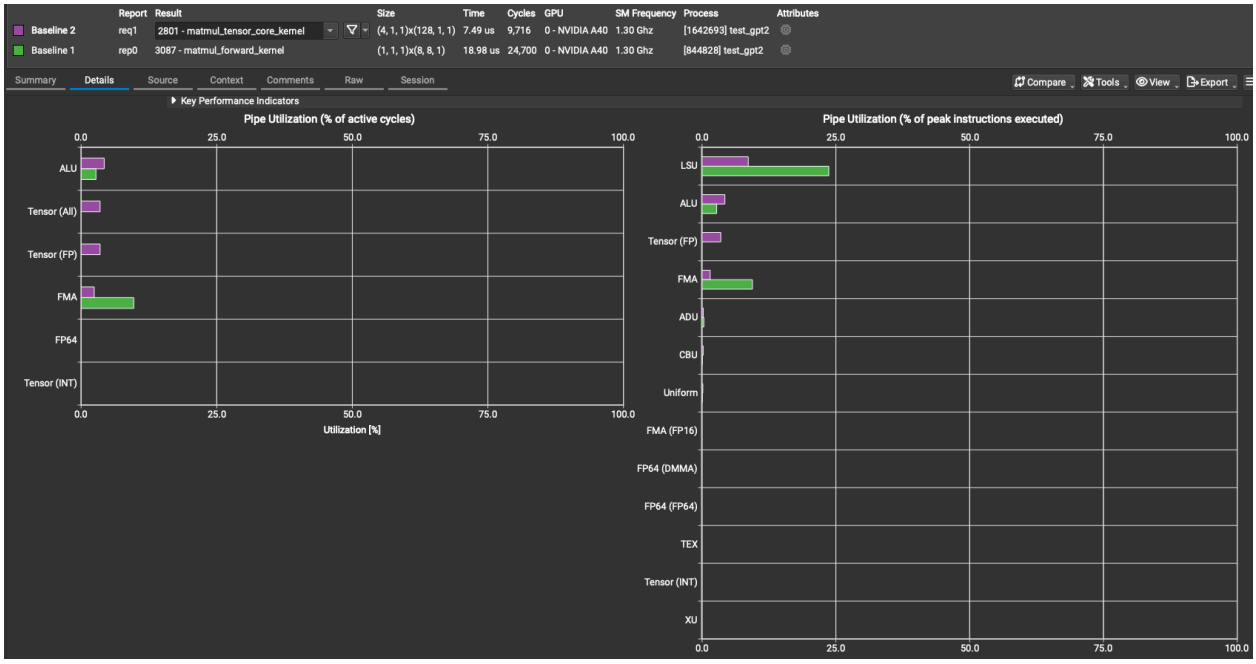


In general this optimization using tensor core beats the joint-register-tiling kernel by a wide margin (~ 2.5 times faster).

Memory & Compute Workload Analysis

In the figure below green bar represent the kernel join tilting register kernel and the purple bar represent the kernel using tensor core. The green (joint register-tiling) kernel shows strong LSU (load/store) activity and a clear FMA bar. That pattern means it's more memory-traffic heavy and executes all its math on the CUDA cores (the FP32 FMA units). In other words, its performance is governed by how quickly it can move data between global/shared memory and registers, and how well it keeps the CUDA cores busy.

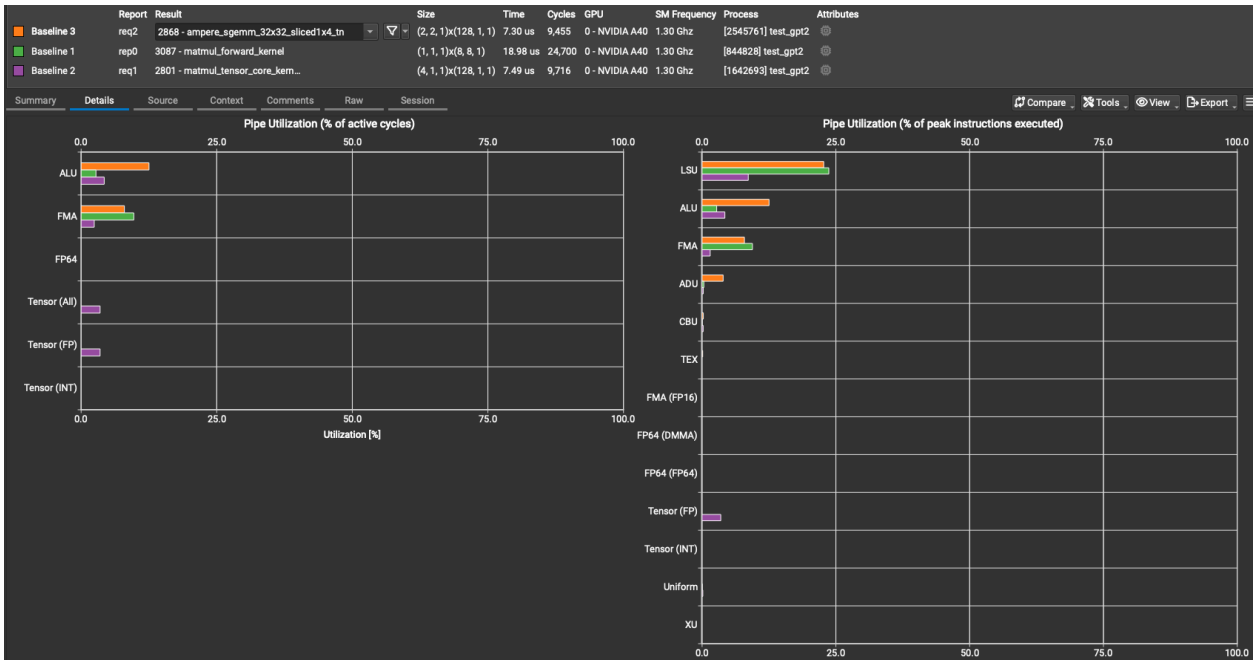
The purple (Tensor Core/WMMA) kernel shows much lower LSU and negligible FMA, which is expected because the Tensor Cores are doing the matrix math instead of the CUDA cores. Each Tensor Core instruction is "chunky": e.g., one `mma.sync.m16n8k8` performs 16×8×8 = 1,024 fused multiply-adds per warp instruction, so even modest Tensor Core activity yields a lot of FLOPs. However, the Tensor(FP) bar is still small, meaning the Tensor Cores aren't fully saturated—likely feed-limited rather than compute-limited.


Report Result Size Time Cycles GPU SM Frequency Process Attributes
Baseline 2 req1 2801 - matmul_tensor_core_kernel (4, 1, 1)x(128, 1, 1) 7.49 us 9,716 0 - NVIDIA A40 1.30 Ghz [1642693] test_gpt2
Baseline 1 rep0 3087 - matmul_forward_kernel (1, 1, 1)x(8, 8, 1) 18.98 us 24,700 0 - NVIDIA A40 1.30 Ghz [844828] test_gpt2
Summary
Details
Source
Context
Comments
Raw
Session
Compare
Tools
View
Export
Key Performance Indicators
Pipe Utilization (% of active cycles)
ALU
Tensor (All)
Tensor (FP)
FMA
FP64
Tensor (INT)
0.0
25.0
50.0
75.0
100.0
Utilization [%]
Pipe Utilization (% of peak instructions executed)
LSU
ALU
Tensor (FP)
FMA
ADU
CBU
Uniform
FMA (FP16)
FP64 (DMMA)
FP64 (FP64)
TEX
Tensor (INT)
XU

Opt #2

The cuBLAS kernel (orange) is the quickest at ~7.30 µs, edging out the Tensor-Core WMMA kernel (~7.49 µs) and beating the register-tiling CUDA-core kernel (~18.98 µs) by ~2.5×.

Memory & Compute Workload Analysis



In this figure, surprisingly, the orange (cuBLAS) kernel‘s Tensor(FP) bar is near zero, so this win comes from a highly tuned FP32 CUDA-core (FMA) path, not from Tensor Cores. we can also see that it also has a high LSU issue rate which indicates that cuBLAS issues lots of loads/stores per cycle.

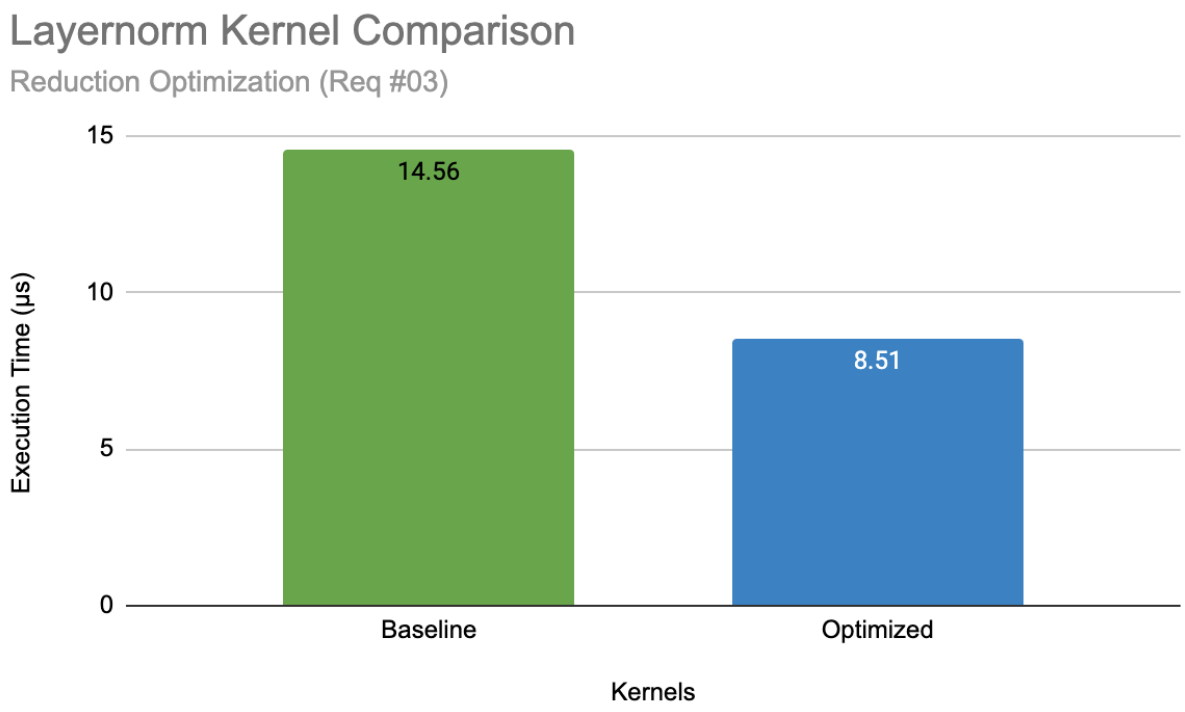
So it seems that the main performance gain stems from cuBLAS doing more and better coordination: wide, coalesced/vectorized accesses; deep prefetch/pipelining (often double/triple-buffered); shared-memory tiling with low bank conflicts; compact ALU/ADU control to advance iterators cheaply;

Because Tensor(FP) is not used, there still much more rooms for improvements, switching to the Tensor-Op path (e.g., `cublasGemmEx(..., CUBLAS_COMPUTE_32F_FAST_TF32, CUBLAS_GEMM_DEFAULT_TENSOR_OP)`) can unlock higher peak FLOPs and typically push runtimes below the current 7.30 µs—while keeping cuBLAS’s best-in-class memory behavior.

Opt #3

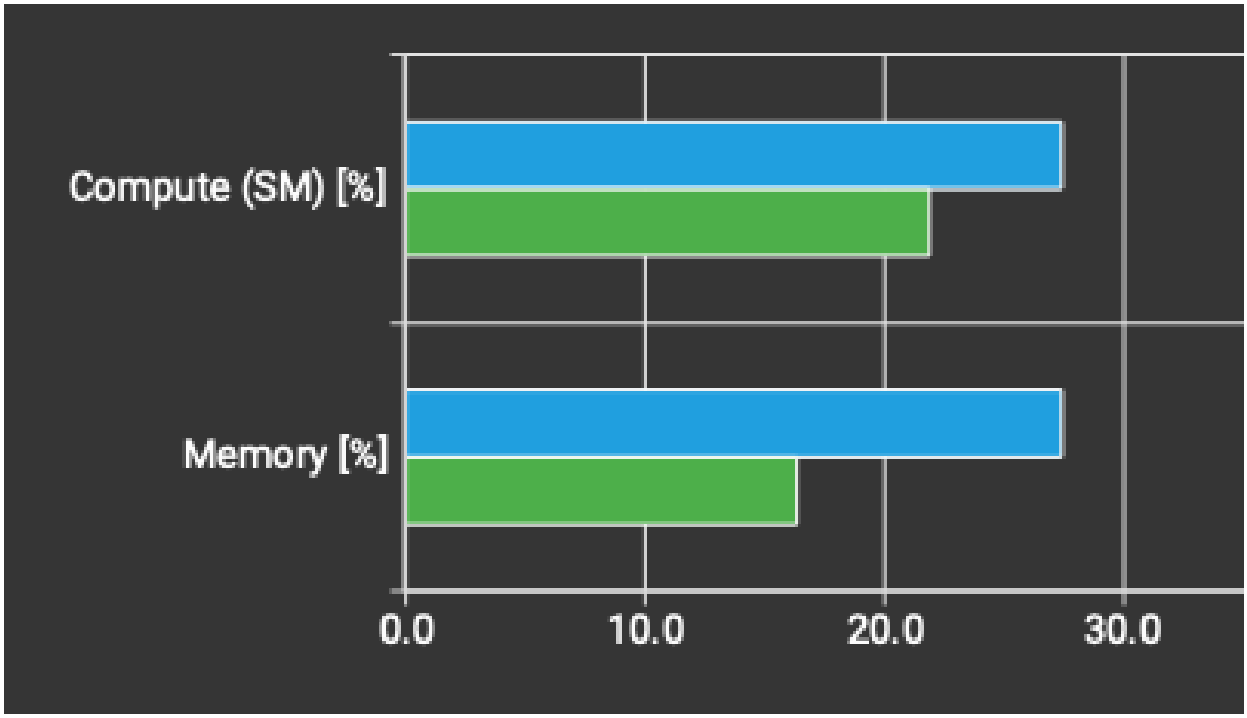
Layernorm

The optimized kernel showed a 6 µs (~42%) reduction in duration, indicating an incredibly high performance boost.

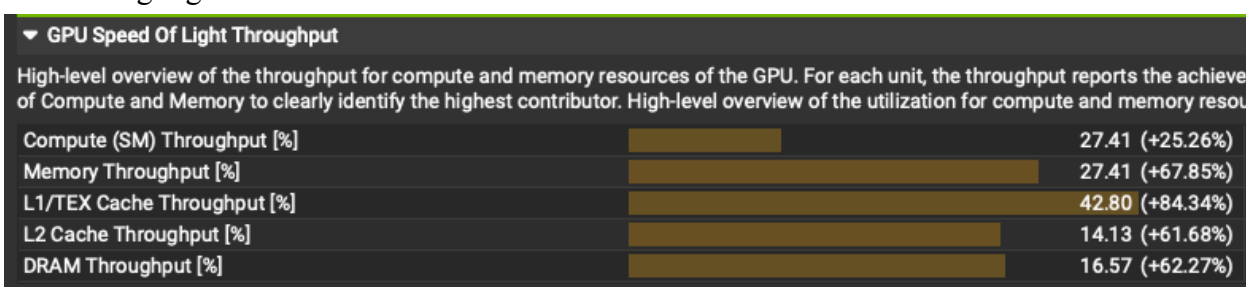


GPU Throughput

While both the baseline (green) and optimized (blue) kernels remain far below their theoretical speed of light (SOL), the optimized kernel showed a 25% and 68% increase in compute and memory throughput respectively.



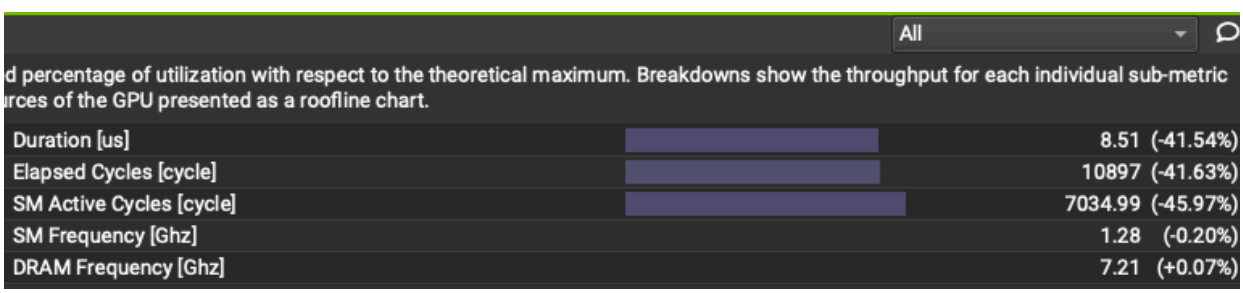
In general, GPU throughput showed significant improvements all around, predominantly for memory over compute. This aligns with expected performance gains since the use of warp level primitives reduces the number of slow global reads and synchronization overhead while maximizing register utilization.



GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieve
of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resou

Compute (SM) Throughput [%]	27.41 (+25.26%)
Memory Throughput [%]	27.41 (+67.85%)
L1/TEX Cache Throughput [%]	42.80 (+84.34%)
L2 Cache Throughput [%]	14.13 (+61.68%)
DRAM Throughput [%]	16.57 (+62.27%)



All

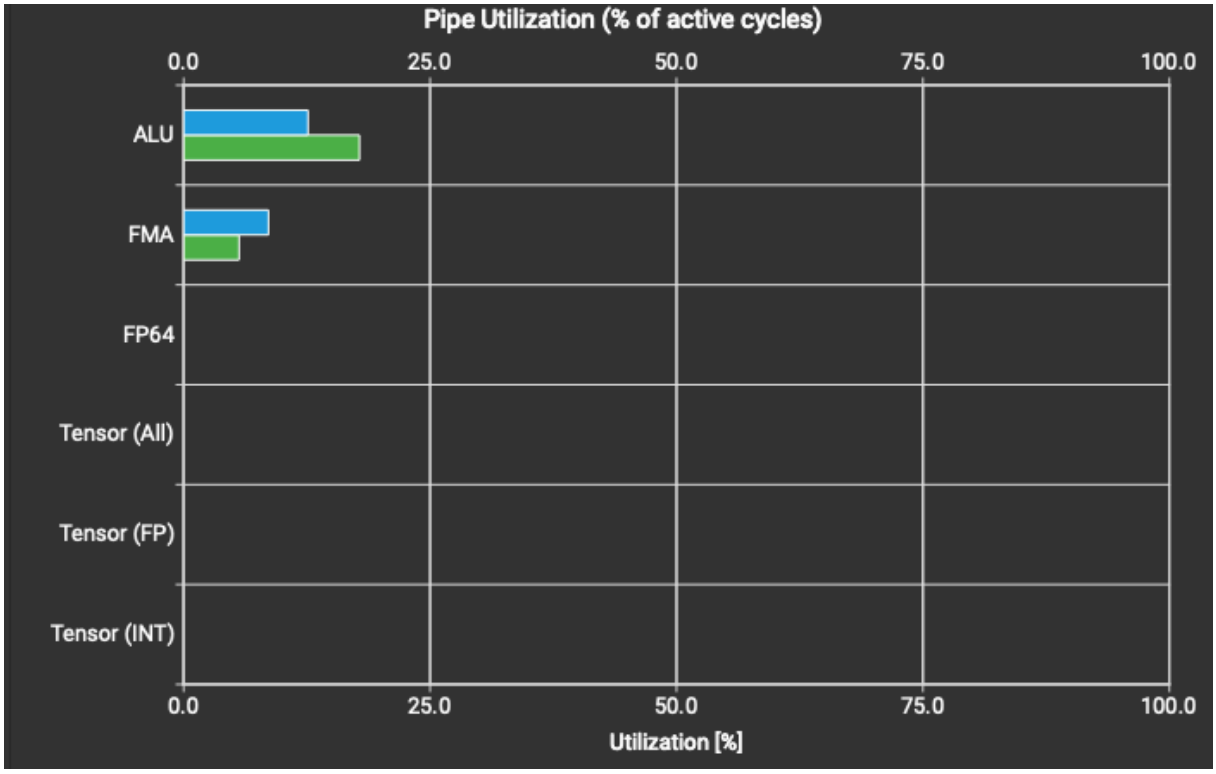
d percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric
rces of the GPU presented as a roofline chart.

Duration [us]	8.51 (-41.54%)
Elapsed Cycles [cycle]	10897 (-41.63%)
SM Active Cycles [cycle]	7034.99 (-45.97%)
SM Frequency [Ghz]	1.28 (-0.20%)
DRAM Frequency [Ghz]	7.21 (+0.07%)

Compute Workload Analysis

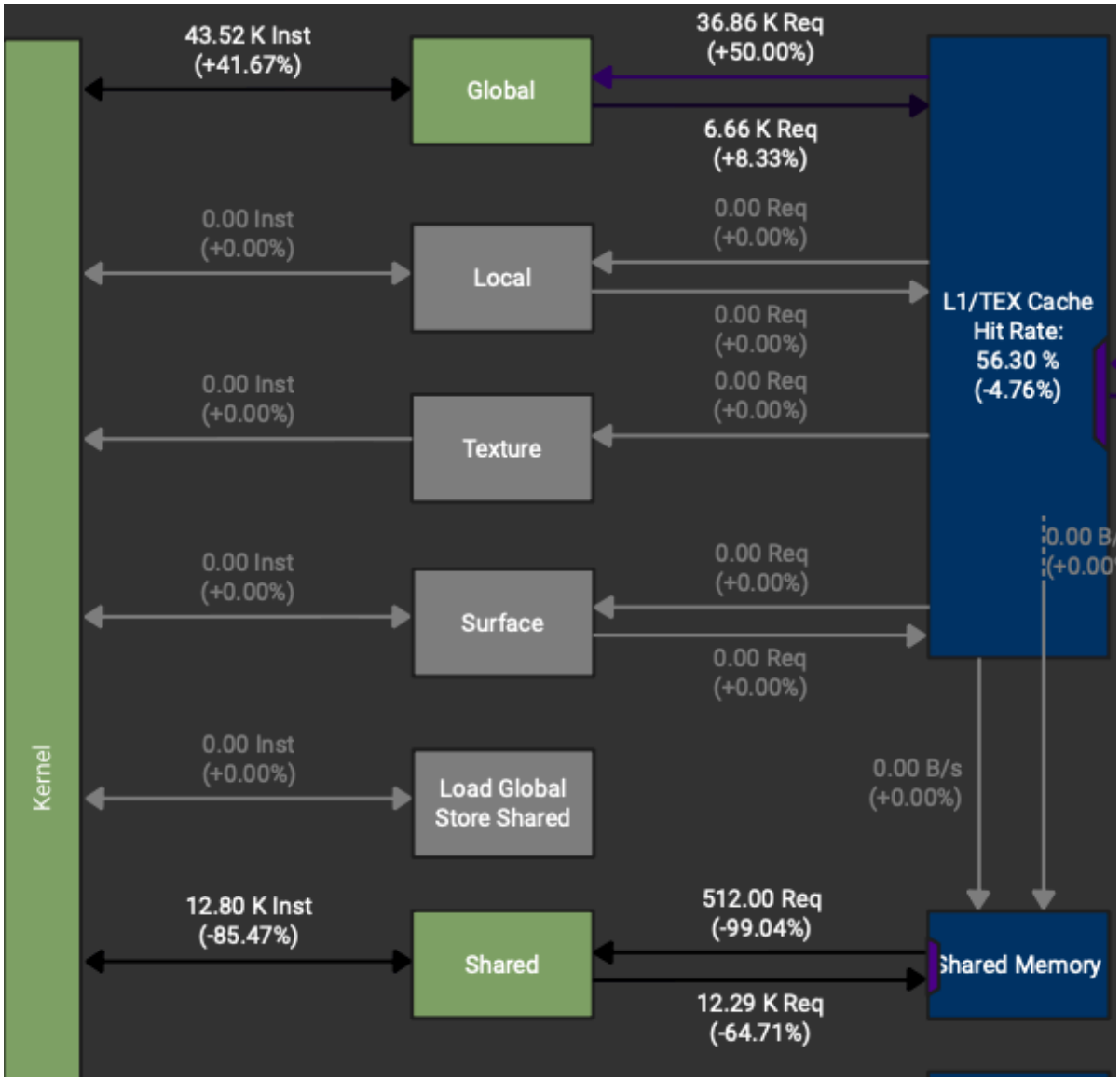
Compared to the baseline (green), the optimized (blue) kernel showed an approximately 30% decrease in ALU utilization and 40% increase in FMA utilization. This is indicative of both the reduction in allocated threads (reduction from 1024 to 768 block size) as well as increased

work efficiency since slower, individual ALU operations are replaced with faster FMA operations.

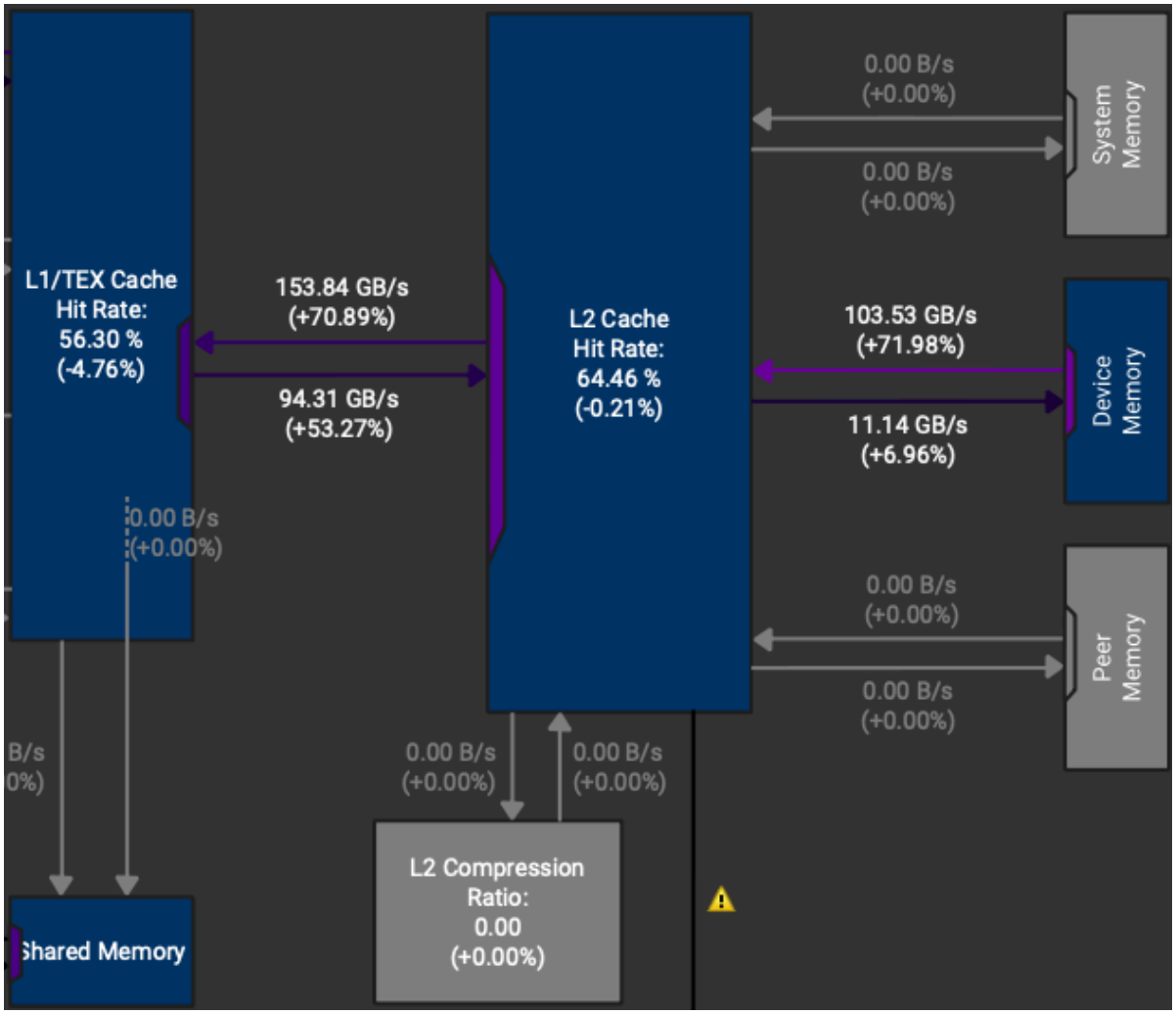


Memory Workload Analysis

Global memory transfers increased significantly, which is likely a reflection of the overall increase in performance gains of the kernel. On the other hand, shared memory transfers decreased. While this may initially be puzzling, it can be explained by the fact that our baseline kernel initially used a block level reduction strategy. Within each block, each thread stored a partial accumulated sum in shared memory, and reduction was executed upon the shared memory array. However, the current kernel performs reduction with warp level primitives, effectively decreasing the number of elements written to shared memory by a factor of 32.



Other memory components such as the L1 & L2 cache showed significant increases in memory bandwidth between each other but also a slight decrease in cache hit rates. This can be explained by an inefficiency in the kernel implementation. Token-level mean and standard deviation reads are stored in global mean and reciprocal standard deviation arrays when they could've been written to shared memory as each block corresponds to a single token.



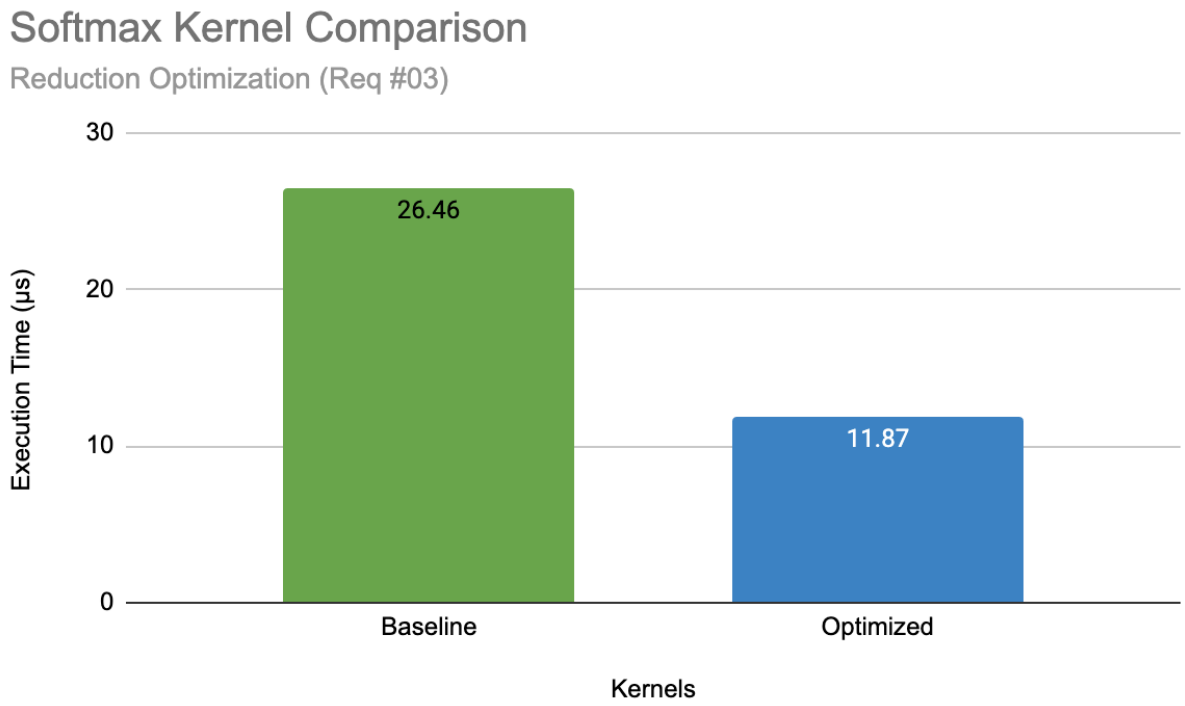
Occupancy

Occupancy showed great gains at ~50%. Maximizing further gains here would involve more fine-tuned scheduling and/or block allocation for the specific structure of our model.

Theoretical Occupancy [%]	100 (+50.00%)
Theoretical Active Warps per SM [warp]	48 (+50.00%)
Achieved Occupancy [%]	87.26 (+46.65%)
Achieved Active Warps Per SM [warp]	41.88 (+46.65%)

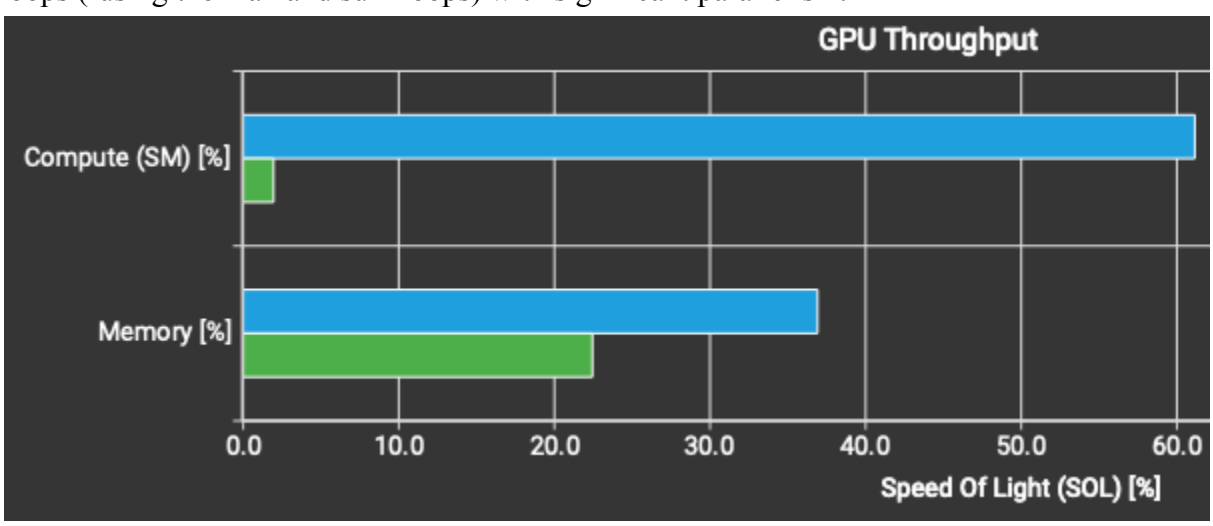
Softmax

The optimized kernel showed a ~15 µs (~55%) reduction in duration, indicating an incredibly high performance boost.



GPU Throughput

The difference between the base and optimized kernels is striking. Memory and compute throughput show a 64% and ~3000% increase respectively. While the contrast in compute throughput is profound, it is reasonable given the context. The baseline kernel sequentially calculates its results across three different for loops, and the optimized kernel only uses two loops (fusing the max and sum loops) with significant parallelism.



Diving deeper, L2 cache throughput decreased by 75% while L1 cache and DRAM increased by 76% and 53% respectively. The decrease in L2 throughput can be explained by the

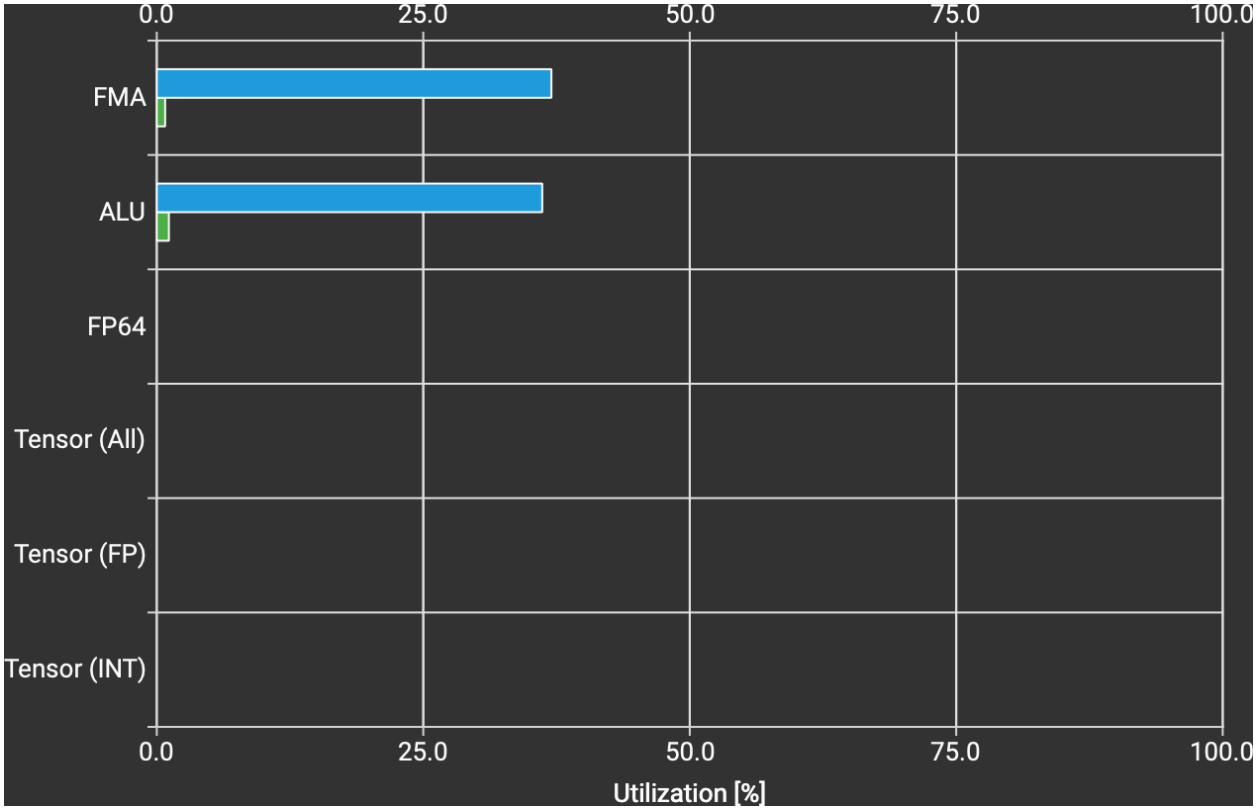
kernel's overall reduction in global memory accesses, which were likely often read from the L2 cache. Consequently, this would also explain why the overall memory throughput did not increase as much as compute.

Compute (SM) Throughput [%]	61.15	(+3023.59%)
Memory Throughput [%]	36.91	(+64.48%)
L1/TEX Cache Throughput [%]	44.91	(+75.85%)
L2 Cache Throughput [%]	5.69	(-74.64%)
DRAM Throughput [%]	8.69	(+52.64%)

Duration [us]	11.87	(-55.14%)
Elapsed Cycles [cycle]	14768	(-57.11%)
SM Active Cycles [cycle]	11971.64	(-44.11%)
SM Frequency [Ghz]	1.24	(-4.52%)
DRAM Frequency [Ghz]	7.21	(-0.19%)

Compute Workload Analysis

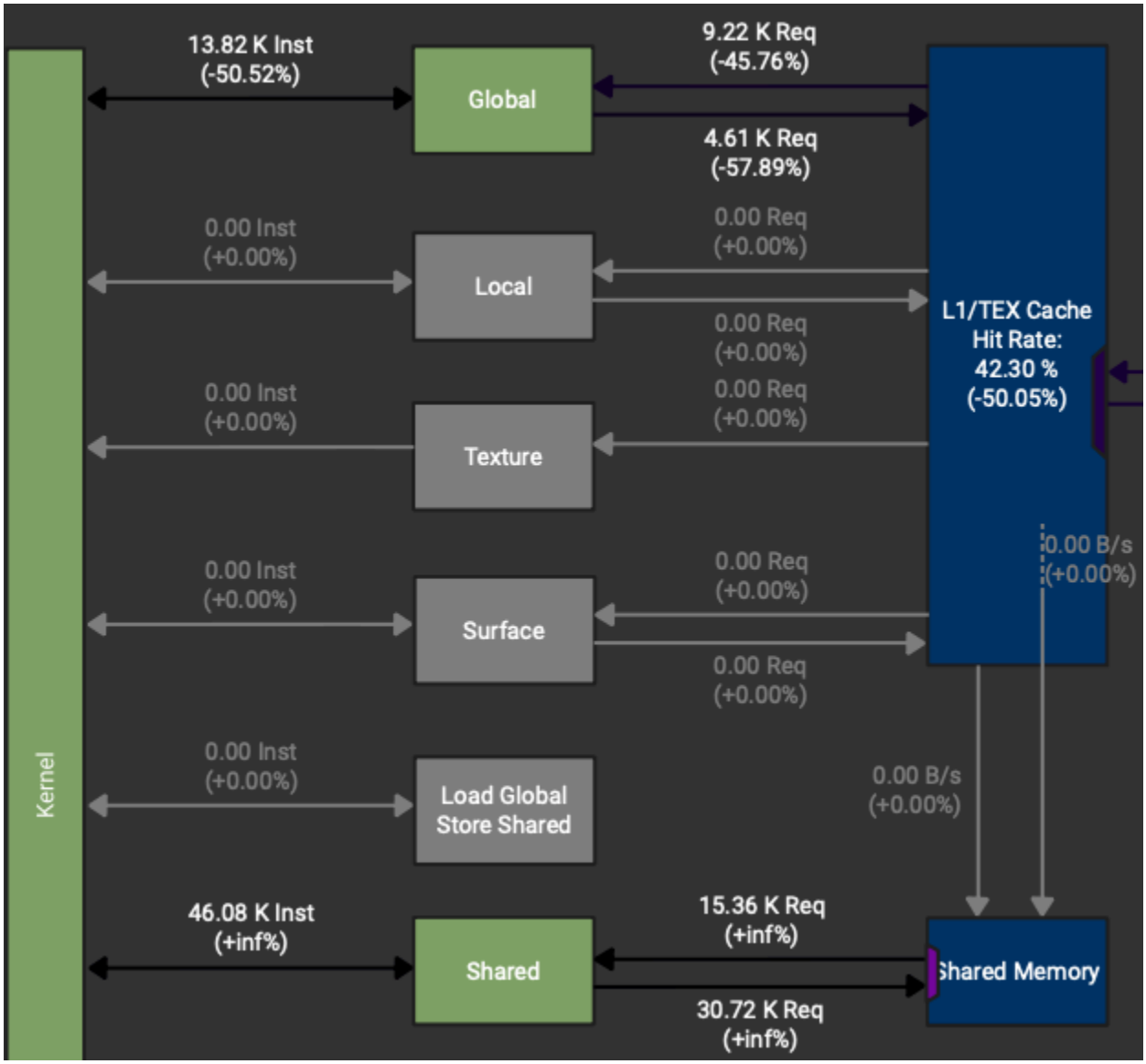
Aligning with previous observations regarding large increases in compute throughput, there's an approximately 30-40x increase in both FMA and ALU pipeline utilization compared to the baseline.



Memory Workload Analysis

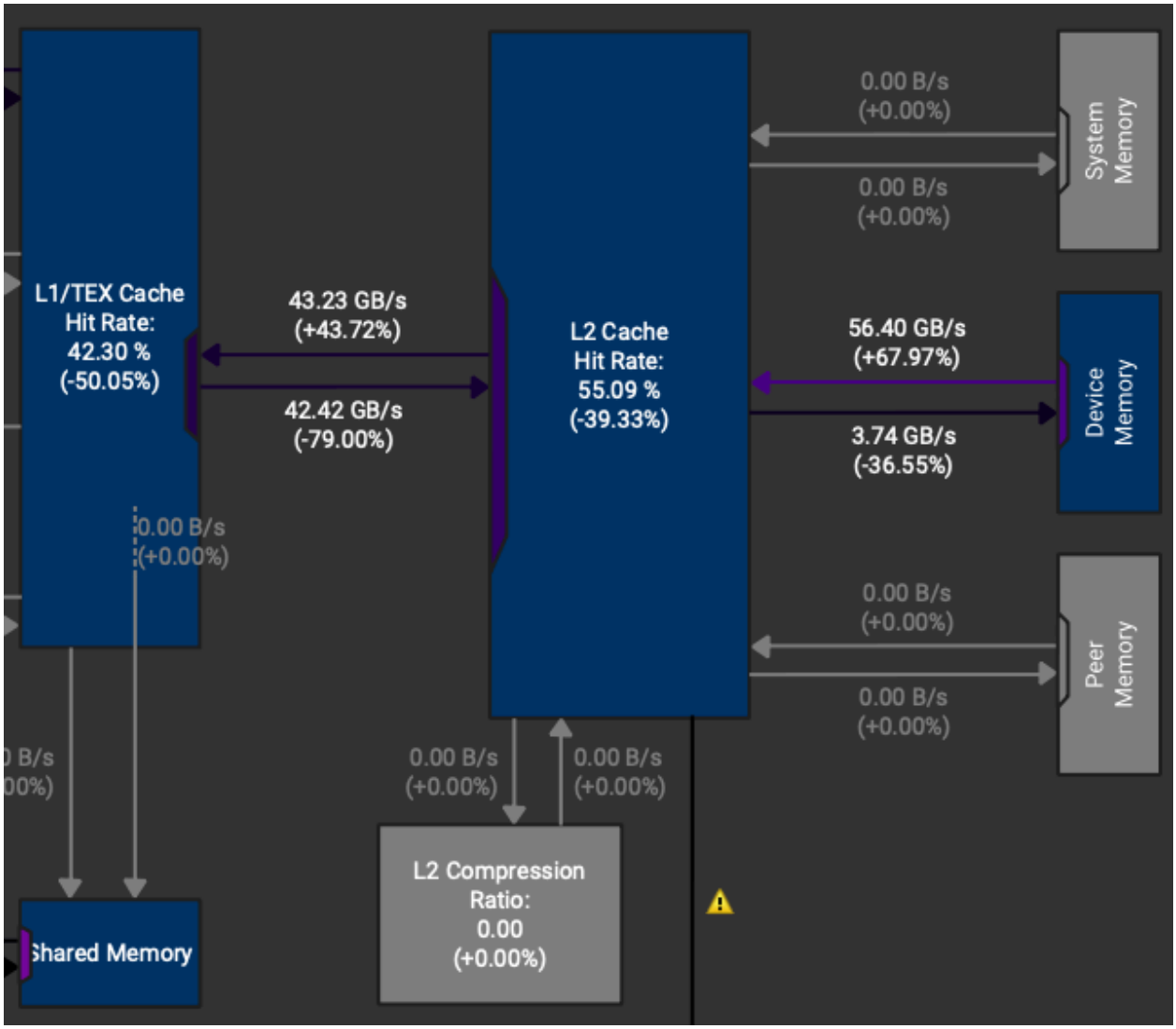
As expected, global memory reads decreased by ~50%. On the flip side, shared memory usage increased by inf %. Given that the baseline kernel did not use any shared memory, this

would explain the infinite increase.



While overall cache hit rates decreased, overall writes to lower levels of memory (i.e. data evictions) also decreased at a far higher rate. The increase in overall throughput could explain why there may be more frequent transfers between memory organization structures as

input values would need to be evicted faster.



Occupancy

Occupancy increased dramatically at a highly similar rate as the increase in compute throughput. While higher occupancy doesn't always result in higher performance, it certainly seems to align closely in this case, especially given how closely the gain in occupancy matches the increase in compute throughput.

Theoretical Occupancy [%]	100	(+200.00%)
Theoretical Active Warps per SM [warp]	48	(+200.00%)
Achieved Occupancy [%]	73.77	(+2974.48%)
Achieved Active Warps Per SM [warp]	35.41	(+2974.48%)

Optimization Proposal

Overview

This proposal outlines a coherent strategy for optimizing our GPT-2 CUDA implementation by integrating three complementary techniques: CUTLASS-based GEMM kernels, AWQ weight quantization, and targeted kernel fusion. The goal is to maximize computational throughput, minimize memory bandwidth bottlenecks, and reduce latency.

The plan involves replacing custom GEMM operations with CUTLASS templates for tensor core utilization, embedding AWQ quantization logic directly into the CUTLASS data path for on-the-fly dequantization, and fusing bandwidth-bound kernels to avoid redundant global memory operations. Together, these optimizations aim to achieve up to 2–4x higher throughput, lower latency, and a 3–4x reduction in memory footprint.

Reasoning

CUTLASS

CUTLASS provides a **highly optimized and customizable** GEMM framework that allows developers to define custom tile shapes, prologues, and epilogues. By using CUTLASS as the backbone for linear operations, the implementation gains the efficiency of tensor cores and shared memory tiling while preserving flexibility, which cuBLAS does **not** offer. It can integrate custom post-processing such as bias addition, activation, and quantization directly within the GEMM epilogue, removing separate kernel launches.

Justification: Replacing hand-tuned GEMMs with CUTLASS offers near-cuBLAS performance while maintaining full control over fusion and quantization.

AWQ

People often do **weight-only quantization** for LLMs because it delivers most of the benefit with far less risk: weights dominate model size and bandwidth, and since they're **static**, you can calibrate once (per-channel/group scales, outlier handling) and keep accuracy close to FP16 while using simple, fast kernels that dequantize weights on the fly and multiply by FP16 activations.

Activation-aware Weight Quantization (AWQ) compresses weights to 4 or 8 bits using asymmetric scaling and outlier correction. In this proposal, weights are quantized offline and stored in a compact format. During inference, custom CUTLASS prologue iterators perform dequantization directly into registers or shared memory before the GEMM computation.

Activations remain in FP16 or FP32 for numerical stability.

Justification: Integrating AWQ inside the GEMM data path removes separate dequantization kernels, hides quantization overhead behind compute, and reduces model size and memory bandwidth requirements by up to 75% while maintaining <1% accuracy degradation.

Custom Kernel Fusions

GPT-2 inference is dominated by **memory-bound** sub-operations such as bias addition, activation, layer normalization, and residual connections as we observed during our profiling. Fusing these operations prevents unnecessary global reads and writes, significantly improving bandwidth utilization. With CUTLASS handling the matrix multiplications, these fused operations can be attached as epilogues or as post-processing kernels that keep intermediate data in registers or shared memory.

Justification: Kernel fusion reduces kernel launch overhead, minimizes memory I/O, and allows intermediate tensors to stay on-chip. This can cut the number of kernel launches per transformer block from 20–30 down to 4–5 and improve end-to-end latency by 1.5–2x.

High Level Plan

1. **Integrate CUTLASS for GEMM operations**
    - Replace existing matrix multiplication kernels with CUTLASS templates configured for tensor core usage.
    - Choose appropriate tile sizes tailored for our GPU architecture.
    - Implement custom epilogues to combine GEMM output, bias addition, and activation.
2. **Embed AWQ Quantization into CUTLASS**
    - Derive AWQ calibration set from GPT-2 test runs
    - Quantize model weights offline using AWQ calibration (and save in 4-bit or 8-bit format.

- Create custom CUTLASS prologue iterators that perform dequantization from packed format to FP16/FP32 before computation.
- Ensure per-channel or per-group scaling factors are efficiently loaded into constant or shared memory.

3. **Apply Targeted Kernel Fusion**
    - **Linear Layers:** Fuse GEMM + bias + activation using CUTLASS epilogues.
    - **Attention Projections:** Combine Q, K, and V projections into a single GEMM with appropriate output striding.
    - **Attention Core:** Integrate FlashAttention-style kernels to fuse QK^T computation with softmax and scaling, using on-chip reductions.
    - **Output and Residual:** Fuse output projection, residual addition, and layer normalization in a single post-processing kernel.
    - **Quantization:** Handle all dequantization within the GEMM prologue and optional re-quantization in the epilogue.
4. **Tune and Extend**
    - Optimize CUTLASS configurations (tile shape, stages, split-K) via profiling.
    - Add constant memory usage for small parameters such as layer norm coefficients.
    - Consider persistent thread blocks for long sequence lengths.
5. **Expected Benefits**
    - Throughput improvement: 2–4x compared to baseline FP16 kernels.
    - Latency reduction: 1.5–2x per transformer block.
    - VRAM reduction: 3–4x smaller model size.